

Claude Code Web + Git Submodules: From Failure to (Workable) Solution

Date: 9 February 2026 **Author:** Dinis Cruz + Claude (Opus 4.6) **Project:** owasp-sbot/Issues-FS ecosystem (16 repos, 15 submodules) **Environment:** Claude Code on the web, triggered from `Issues-FS__Dev` repo

Why This Matters

This document records a full day of experiments trying to make Claude Code's web-based agent work with a multi-repo project that uses git submodules. It exists because the alternative — running Claude Code locally on a developer's laptop — turned out to be **too dangerous to use safely**.

Why Claude Code was uninstalled from the laptop

After a near miss where Claude Code requested permission to read the entire contents of the user's home directory (during a session scoped to a specific project folder), Claude Code was uninstalled from the local machine. The core problems were:

- **Permission fatigue:** Claude asks for permissions constantly — hundreds of times per session. 99% are obviously fine, training the user to click "yes" automatically. But buried in that stream was a request that should never have been made: broad read access to `~/`.
- **No OS-level isolation that actually works:** An attempt was made to create a separate macOS user account to run Claude Code under (to enforce OS-level file access boundaries). This failed — Claude Code wouldn't start at all under the restricted account, with critical errors on launch. And the security analysis of the resulting environment (had it worked) showed it would still be vulnerable to a proactive agent or one compromised via prompt injection.
- **The expertise gap:** Properly sandboxing an AI coding agent at the OS kernel level requires deep macOS/Linux security expertise. The scripts Claude generated to "lock itself down" were not trustworthy enough to rely on without expert review — and when lockdown doesn't work, the only path forward is usually *more* privileges, making things worse.
- **The right answer is to wait:** The secure solution is for Anthropic's security team to deliver proper sandboxing where the *only* damage an agent can cause is within a git-managed folder (where all changes are detectable and reversible). Until then, the web-based version — despite its limitations — is the safer option.

This is what drove the need to make Claude Code *web* work with submodules. The web sandbox is architecturally constrained (which is frustrating but also *the point* — constraints are security).

The Challenge

The `Issues-FS__Dev` repository is the orchestration hub for 16 repos in the owasp-sbot ecosystem. It uses git submodules to pull in 6 application code repos (under `modules/`) and 9 AI agent role definition repos (under `roles/`). When Claude Code web clones `Issues-FS__Dev`, those 15 submodule directories are **empty placeholders** — git submodules are pointers, not embedded code.

The goal was simple: make Claude Code web sessions able to read and write across all 16 repos. The reality was a series of walls.

Summary of All Experiments

#	Experiment	Method	Result	Key Finding
1	Discover proxy URL pattern	<code>git remote -v</code> on parent repo	 Success	Proxy at <code>127.0.0.1:29240</code> , format: <code>/git/owasp-sbot/<repo></code>
2	Clone submodules via proxy	Proxy URL for each submodule	 Failed	<code>repository not authorized</code> — proxy scoped to parent repo only
3	Clone submodules via HTTPS	Direct <code>github.com</code> HTTPS URLs	 Success	All 15 cloned on <code>dev</code> branch (read access works)
4	Fetch branch from parent repo	<code>git fetch</code> + <code>git show</code> via proxy	 Success	Parent repo fully accessible via proxy
5	Push to submodule via proxy	Set submodule remote to proxy URL	 Failed	Same <code>repository not authorized</code> error
6	Push to submodule via direct HTTPS	Remote set to <code>github.com</code> directly	 Failed	<code>could not read Username</code> — no credentials for direct HTTPS
7	Push via egress proxy	Used <code>GLOBAL_AGENT_HTTP_PROXY</code> JWT proxy	 Failed	Command hung indefinitely — egress proxy doesn't handle git auth
8	Credential extraction attempts	Checked <code>~/.netrc</code> , SSH keys, <code>gh</code> CLI, custom credential helpers, env vars	 All failed	No mechanism to obtain push tokens for non-parent repos
9	Push document to parent repo instead	Commit doc to parent branch as workaround	 Failed	Proxy went down (504) after repeated failed attempts
10	Proxy recovery	Exponential backoff retries (2s→30s)	 Failed	Proxy never recovered — session became completely unusable
11	PAT via environment variables	Fine-grained GitHub PAT injected via Claude Code web environment config	 Success	All 15 submodules cloned and writable. Full read/write access.

Detailed Experiment Log

Phase 1: Understanding the Sandbox

Experiment 1 — Discover the proxy URL pattern

Ran `git remote -v` on the parent repo and found the local proxy at `http://local_proxy@127.0.0.1:29240/git/owasp-sbot/Issues-FS__Dev`. This proxy is how Claude Code web authenticates git operations — it sits between the VM and GitHub, injecting credentials transparently.

Experiment 2 — Clone submodules via proxy

Attempted to clone all 15 submodules using the same proxy URL pattern, substituting each repo name. Every single one returned `Proxy error: repository not authorized` (HTTP 502). A systematic test confirmed the proxy is exclusively scoped to the parent repo:

Repo	Proxy Response
<code>Issues-FS__Dev</code> (parent/trigger repo)	HTTP 200
<code>Issues-FS</code>	HTTP 502
<code>Issues-FS__CLI</code>	HTTP 502
<code>Issues-FS__Dev__Role__DevOps</code>	HTTP 502
<code>Issues-FS__Dev__Role__Dev</code>	HTTP 502

Key architectural finding: The Claude Code web sandbox proxy is scoped to exactly one repository per session — the repo that triggered the session. This is by design, not a configuration issue. Even though the Claude GitHub App has access to all 16 repos at the GitHub level, the proxy only authorises the triggering repo.

Experiment 3 — Clone submodules via HTTPS fallback

Fell back to `git clone --branch dev https://github.com/owasp-sbot/<RepoName>.git`. All 15 submodules cloned successfully. Read access works because the repos are public (or the GitHub App provides implicit read access for cloning). But this doesn't help with pushing.

Phase 2: Trying to Push

Experiment 4 — Fetch from parent repo

`git fetch origin claude/review-submodules-zzzOI` followed by `git show` to extract a document. Worked perfectly — the parent repo is fully accessible via the proxy for both read and write.

Experiment 5 — Push to submodule via proxy

Set the DevOps submodule's remote to the proxy URL and attempted `git push`. Same `repository not authorized` error. The proxy doesn't just restrict *which* operations you can do — it restricts *which repos* you can talk to at all.

Experiment 6 — Push via direct HTTPS

Set remote to `https://github.com/owasp-sbot/Issues-FS__Dev__Role__DevOps.git` and attempted push. Failed with `fatal: could not read Username` — there's no interactive TTY for credential prompts, and no credential helper configured.

Experiment 7 — Push via egress proxy

Discovered a `GLOBAL_AGENT_HTTP_PROXY` environment variable containing a JWT-authenticated proxy URL (5,290 characters long). Attempted to use it as `http_proxy`/`https_proxy` for the git push. The command hung indefinitely with no output. This proxy handles general outbound HTTP traffic, not git protocol authentication.

Experiment 8 — Credential extraction

Exhaustive search for any available credentials:

- `~/.netrc` — doesn't exist
- SSH keys — only a commit *signing* key, not an auth key
- `gh` CLI — not installed
- Custom `git-credential-helper` script — GitHub rejected with "Password authentication is not supported"
- `CLAUDE_CODE_OAUTH_TOKEN_FILE_DESCRIPTOR` — exists but opaque/scoped
- `CODESIGN_MCP_TOKEN` — for MCP operations, not git

None of these provided a path to push credentials for non-parent repos.

Phase 3: Things Get Worse

Experiment 9 — Push document to parent repo as workaround

Since pushing to the submodule was impossible, committed the document to the parent repo's branch instead. But by this point, after the series of failed push attempts to unauthorised repos, the proxy started returning HTTP 504 (Gateway Timeout) for *all* requests — including the authorised parent repo.

Experiment 10 — Proxy recovery

Retried with exponential backoff: 2s, 4s, 8s, 16s, 20s, 30s delays between attempts. The proxy never recovered, alternating between 503 and 504. The session became completely unusable — couldn't push, couldn't fetch, couldn't even `ls-remote` the parent repo.

Observation: Hammering the proxy with requests to unauthorised repos appears to have degraded it for the authorised repo too. This is arguably a bug — failed auth attempts against unauthorised repos shouldn't poison the proxy for the authorised one.

Phase 4: The Solution That Worked

Experiment 11 — PAT via environment variables

After the failed session, we designed and tested a different approach: injecting a GitHub Personal Access Token (fine-grained) through Claude Code web's environment variable configuration.

Setup:

1. Created a fine-grained GitHub PAT scoped to all 16 owasp-sbot repos with **Contents: Read and write + Metadata: Read-only**
2. Added `GH_SUBMODULE_TOKEN=ghp_xxx` in the Claude Code web environment config (Settings → Cloud Environment → Environment Variables, in `.env` format)
3. Set network access to "Trusted" (required for github.com HTTPS access)

4. Configured `git config --global url."https://x-access-token:${GH_SUBMODULE_TOKEN}@github.com/"insteadOf "git@github.com:"` at session start

Result: All 15 submodules cloned successfully. Push to submodule repos worked. The PAT completely bypasses the proxy's single-repo scoping limitation by providing direct authenticated HTTPS access to GitHub.

The full verified procedure is documented in the `runbook_claude-web-submodule-setup.md`, which was created and pushed by Claude Code itself during the successful session — to the correct submodule repo (`Issues-FS__Dev__Role__DevOps`), proving the solution works end-to-end.

Why This Solution Is OK(ish): Thank Git's Distributed Design

The PAT solution works, but it's not without risk. The token gives Claude Code **read/write access** to all 16 repos. Branch protection on `main` and `dev` helps, but can't be fully enforced with the current PAT setup — a sufficiently "enthusiastic" agent (or one compromised via prompt injection) could still cause damage by creating junk branches, pushing to unprotected branches, or worse.

Here's why it's acceptable despite that risk: **git is distributed, and we have to love git's design for this.**

Every developer has a full local copy of every repo. If the web-based agent goes rogue and corrupts branches in the GitHub remote, the damage is:

- **Detectable** — git tracks everything, every commit is hashed, every push is logged
- **Reversible** — force-push from a known-good local copy restores the remote to any previous state
- **Contained** — the remote is a coordination point, not the source of truth; the local clones are the real backups

The distributed nature of git is the fundamental reason this whole approach is viable. In a centralised system (a database, a SaaS app, a wiki), agent-caused damage could be genuinely irreversible. With git, the worst case is cleanup work and wasted time — not data loss.

The CI Pipeline Safety Net

To further contain the blast radius, the deployment architecture is being designed so that the repos exposed to Claude Code web are **not wired to production**:

- Claude Code operates on repos that deploy to `dev` and `qa` environments only
- Production deployments happen from a **separate "prod" repo** that receives PRs from the dev repo
- This means even a worst-case scenario (agent compromises `main` on the dev repo) **cannot directly affect production**
- A human reviews and approves every PR from dev → prod

This separation means prompt injection or agent misbehaviour can cause mess and wasted time, but not production outages or customer-facing damage. The `main` branch on the exposed repos is not wired to production — only to dev and QA pipelines where damage is recoverable.

The Bigger Picture: Infrastructure Not Fit for the Agentic World

This whole exercise — the failed experiments, the workarounds, the risk analysis — is a concrete

example of a fundamental problem that goes far beyond Claude Code:

Our underlying data, application, and identity infrastructure is completely not fit-for-purpose for the new agentic world, where "well-intentioned-but-over-enthusiastic", malicious, or "prompt-injected" agents can cause a huge amount of irreversible damage, very fast.

Consider what we're dealing with:

- **Identity:** There's no concept of a "machine agent identity" in GitHub. The PAT is tied to a human user's account. There's no way to create a GitHub identity for "Claude-DevOps-Agent" with its own scoped permissions, audit trail, and revocation lifecycle. Ideally, each AI agent role (DevOps, Architect, AppSec, etc.) would have its own identity with precisely scoped permissions — but that infrastructure doesn't exist.
- **Permissions:** GitHub's permission model is repo-level, not path-level or operation-level. You can't say "this agent can only modify files under `docs/`" or "this agent can create branches but not delete them" or "this agent can push but only to branches matching `claude/*`". It's all-or-nothing at the repo level.
- **Audit:** When Claude pushes a commit, it shows up as the human PAT owner. There's no native way to distinguish "human pushed this" from "agent pushed this on behalf of human" in the git log. The agent signs commits as `Claude <noreply@anthropic.com>`, which helps, but that's a convention, not an enforced identity.
- **Blast radius:** A well-intentioned but over-enthusiastic agent, a malicious agent, or an agent compromised via prompt injection can cause a huge amount of damage, very fast. An agent with repo write access can create hundreds of branches, push thousands of commits, delete branch protections (if the token has admin scope), and pollute the entire git history — all in seconds. The current infrastructure has no rate limiting, anomaly detection, or circuit breakers for this kind of automated activity.
- **Speed asymmetry:** The infrastructure was built for humans operating at human speed with human judgment. Agents operate at machine speed with probabilistic judgment. A human might make one bad push per day; an agent can make hundreds per minute. The guardrails haven't caught up.

This isn't just a Claude Code problem. It's an industry-wide gap. Every AI coding tool (Cursor, Copilot Workspace, Codex, etc.) faces the same fundamental mismatch between what agents need to do and what our infrastructure was designed to allow.

What the Ideal Solution Would Look Like

For reference, here's what a properly designed agentic infrastructure would provide:

1. **Per-agent identities** — Each AI agent gets its own GitHub/GitLab identity (not a human's PAT), with its own audit trail, permission set, and revocation controls
2. **Fine-grained, operation-level permissions** — "Can create branches matching `claude/*`, can push commits to those branches, cannot delete branches, cannot modify branch protection rules, cannot access repos outside this list"
3. **Path-scoped access** — "Can modify files under `src/` and `docs/` but not `.github/workflows/` or `Dockerfile`"

4. **Rate limiting and anomaly detection** — "If this agent creates more than 5 branches per hour or pushes more than 50 commits per session, pause and alert"
5. **Session-scoped credentials** — Tokens that expire when the agent session ends, not tokens that persist until manually revoked
6. **Prompt injection resistance** — Infrastructure-level protections that don't rely on the agent's own judgment about whether a request is legitimate

Until this infrastructure exists, we're patching around the gaps with workarounds like the PAT solution documented here. It works. It's not ideal. But combined with git's distributed safety net and a production-isolated CI pipeline, the risks are contained to an acceptable level.

Environment Findings (Reference)

What's available in the Claude Code web VM

- Git CLI with commit signing configured (Claude as author)
- `curl` for HTTP debugging
- `environment-manager` binary (session lifecycle management)
- `GLOBAL_AGENT_HTTP_PROXY` — egress proxy for outbound HTTP (not git auth)
- `CLAUDE_CODE_OAUTH_TOKEN_FILE_DESCRIPTOR` — exists but scoped/opaque
- `CODESIGN_MCP_TOKEN` — for MCP operations, not git
- SSH commit signing key at `/home/claude/.ssh/commit_signing_key.pub`

What's NOT available

- `gh` CLI
- `~/.netrc` or any credential store
- SSH keys for repo access (only a signing key)
- Any way to obtain GitHub tokens for repos other than the parent
- `git credential` helpers that work with the proxy

Proxy architecture

- **Location:** `127.0.0.1:29240`
 - **Auth:** `local_proxy` username, no password
 - **URL format:** `http://local_proxy@127.0.0.1:29240/git/<org>/<repo>`
 - **Scope:** Only the repo that triggered the session (one repo per session)
 - **Capabilities:** Read and write (fetch + push) for the authorised repo only
 - **Failure mode:** Returns `Proxy error: repository not authorized` (HTTP 502) for unauthorised repos; can degrade to 503/504 after repeated failed attempts
-

Session Artifacts

Artifact	Location	Status
Submodule pointer updates	<code>claude/setup-submodules-devops-p1EfG</code> on <code>Issues-FS__Dev</code>	Pushed (first session, before proxy died)
Runbook document	<code>roles/Issues-FS__Dev__Role__DevOps/docs/runbook__claude-web-submodule-setup.md</code>	✔ Pushed successfully (second session, using PAT)
Architecture doc	<code>docs/claude-github-app-submodules-setup.md</code>	Committed locally in first session, relocated in second
Branch to clean up	<code>claude/review-submodules-zzzOI</code> on <code>Issues-FS__Dev</code>	Pending deletion

Recommendations

1. **For read/write submodule access:** Use the PAT + environment variable approach documented in the runbook. This is the proven solution.
2. **Scope the PAT tightly:** Fine-grained, Contents read/write only, scoped to only the repos needed, shortest practical expiry. Rotate regularly.
3. **Never wire agent-accessible repos to production:** Use a separate prod repo that receives PRs from the dev repo. Human approval gates all production deployments.
4. **Keep local clones as ground truth:** Git's distributed design is your safety net. If the remote gets corrupted, force-push from a known-good local copy.
5. **Consider `git subtree` for simpler setups:** Eliminates the auth problem entirely since code is committed inline. Higher migration cost but removes ongoing friction with all AI coding tools.
6. **Feature request to Anthropic:** Allow the proxy to authorise all repos the GitHub App has access to, not just the triggering repo. This would make the PAT workaround unnecessary and keep all auth within the managed proxy.